

Exercise 1: Control Structures

Scenario 1: The bank wants to apply a discount to loan interest rates for customers above 60 years old.

- **Question:** Write a PL/SQL block that loops through all customers, checks their age, and if they are above 60, apply a 1% discount to their current loan interest rates.

Solution:

Creating table Customers:

```
CREATE TABLE Customers (  
  CustomerID NUMBER PRIMARY KEY,  
  Name VARCHAR2(50),  
  Age NUMBER  
);  
  
CREATE TABLE Loans (  
  LoanID NUMBER PRIMARY KEY,  
  CustomerID NUMBER,  
  InterestRate NUMBER(5,2),  
  FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID)  
);
```

Inserting Data:

```
INSERT INTO Customers VALUES (1, 'John', 65);  
INSERT INTO Customers VALUES (2, 'Alice', 45);  
INSERT INTO Customers VALUES (3, 'Robert', 70);  
  
COMMIT;  
  
INSERT INTO Loans VALUES (101, 1, 8.50);  
INSERT INTO Loans VALUES (102, 2, 9.00);  
INSERT INTO Loans VALUES (103, 3, 7.50);  
  
COMMIT;
```

Execte:

```

BEGIN

FOR c IN (SELECT CustomerID, Age FROM Customers)
LOOP

    IF c.Age > 60 THEN

        UPDATE Loans
        SET InterestRate = InterestRate - 1
        WHERE CustomerID = c.CustomerID;

    END IF;

END LOOP;

END;
/

```

```
SELECT * FROM Loans;
```

Output:

Query result	Script output	DBMS output	Explain Plan	SQL history
  Download ▼ Execution time: 0.157 seconds				
	LOANID	CUSTOMERID	INTERESTRATE	
1	101	1	7.5	
2	102	2	9	
3	103	3	6.5	

Scenario 2: A customer can be promoted to VIP status based on their balance.

Question: Write a PL/SQL block that iterates through all customers and sets a flag IsVIP to TRUE for those with a balance over \$10,000.

```

ALTER TABLE Customers
ADD (
    Balance NUMBER,
    IsVIP VARCHAR2(5)
)

```

```
);
```

```
UPDATE Customers  
SET Balance = 12000  
WHERE CustomerID = 1;
```

```
UPDATE Customers  
SET Balance = 8000  
WHERE CustomerID = 2;
```

```
UPDATE Customers  
SET Balance = 15000  
WHERE CustomerID = 3;
```

```
COMMIT;
```

```
BEGIN  
  
FOR c IN (SELECT CustomerID, Balance FROM Customers)  
LOOP  
  
    IF c.Balance > 10000 THEN  
  
        UPDATE Customers  
        SET IsVIP = 'TRUE'  
        WHERE CustomerID = c.CustomerID;  
  
    END IF;  
  
END LOOP;  
  
END;  
/
```

```
SELECT * FROM Customers;
```

Output:

Query result

Script output

DBMS output

Explain Plan

SQL history

Download

Execution time: 0.008 seconds

	CUSTOMERID	NAME	AGE	BALANCE	ISVIP
1	1	John	65	12000	TRUE
2	2	Alice	45	8000	(null)
3	3	Robert	70	15000	TRUE

Scenario 3: The bank wants to send reminders to customers whose loans are due within the next 30 days.

- **Question:** Write a PL/SQL block that fetches all loans due in the next 30 days and prints a reminder message for each customer.

```
ALTER TABLE Loans
ADD DueDate DATE;
```

```
UPDATE Loans
SET DueDate = SYSDATE + 15
WHERE LoanID = 101;
```

```
UPDATE Loans
SET DueDate = SYSDATE + 60
WHERE LoanID = 102;
```

```
UPDATE Loans
SET DueDate = SYSDATE + 20
WHERE LoanID = 103;
```

```
COMMIT;
```

```
BEGIN

FOR I IN (
  SELECT LoanID, CustomerID, DueDate
  FROM Loans
  WHERE DueDate BETWEEN SYSDATE AND SYSDATE + 30
)
```

```

LOOP

    DBMS_OUTPUT.PUT_LINE(
        'Reminder: Customer ' || l.CustomerID ||
        ' has Loan ' || l.LoanID ||
        ' due on ' || l.DueDate
    );

END LOOP;

END;
/

```

Output:

Query result	Script output	DBMS output	Explain Plan	SQL history
  Reminder: Customer 1 has Loan 101 due on 15-JUL-26 Reminder: Customer 3 has Loan 103 due on 20-JUL-26 PL/SQL procedure successfully completed. Elapsed: 00:00:00.015				

Exercise 3: Stored Procedures

Scenario 1: The bank needs to process monthly interest for all savings accounts.

- **Question:** Write a stored procedure **ProcessMonthlyInterest** that calculates and updates the balance of all savings accounts by applying an interest rate of 1% to the current balance.

```

CREATE TABLE SavingsAccounts (

```

```
AccountID NUMBER PRIMARY KEY,  
CustomerName VARCHAR2(50),  
Balance NUMBER(10,2)  
);
```

Inserting data:

```
INSERT INTO SavingsAccounts VALUES (1,'John',10000);  
INSERT INTO SavingsAccounts VALUES (2,'Alice',15000);  
INSERT INTO SavingsAccounts VALUES (3,'Robert',20000);  
  
COMMIT;
```

Create stored Procedure:

```
CREATE OR REPLACE PROCEDURE ProcessMonthlyInterest  
IS  
BEGIN  
    UPDATE SavingsAccounts  
    SET Balance = Balance + (Balance * 0.01);  
  
    COMMIT;  
END;  
/
```

Execute:

```
BEGIN  
    ProcessMonthlyInterest;  
END;  
/
```

Checking:

```
SELECT * FROM SavingsAccounts;
```

Output:

Query result Script output DBMS output Explain Plan SQL history				
  Download ▾ Execution time: 0.014 seconds				
	ACCOUNTID	CUSTOMERNAME	BALANCE	
1	1	John	10100	
2	2	Alice	15150	
3	3	Robert	20200	

Scenario 2: The bank wants to implement a bonus scheme for employees based on their performance.

- **Question:** Write a stored procedure **UpdateEmployeeBonus** that updates the salary of employees in each department by adding a bonus percentage passed as a parameter.

Creating Table:

```
CREATE TABLE Employees (
  EmployeeID NUMBER PRIMARY KEY,
  EmployeeName VARCHAR2(50),
  Department VARCHAR2(30),
  Salary NUMBER(10,2)
);
```

Inserting Data:

```
INSERT INTO Employees VALUES (1,'Rahul','IT',50000);
INSERT INTO Employees VALUES (2,'Priya','HR',45000);
INSERT INTO Employees VALUES (3,'Amit','IT',60000);

COMMIT;
```

Create Procedure:

```
CREATE OR REPLACE PROCEDURE UpdateEmployeeBonus(
  p_Department IN VARCHAR2,
  p_BonusPercent IN NUMBER
)
IS
BEGIN
  UPDATE Employees
```

```

SET Salary = Salary + (Salary * p_BonusPercent / 100)
WHERE Department = p_Department;

COMMIT;
END;
/

```

Execute:

```

BEGIN
    UpdateEmployeeBonus('IT',10);
END;
/

```

Checking:

```
SELECT * FROM Employees;
```

Output:

Query result	Script output	DBMS output	Explain Plan	SQL history
  Download ▾ Execution time: 0.009 seconds				
	EMPLOYEEID	EMPLOYEENAME	DEPARTMENT	SALARY
1	1	Rahul	IT	55000
2	2	Priya	HR	45000
3	3	Amit	IT	66000

Scenario 3: Customers should be able to transfer funds between their accounts.

- **Question:** Write a stored procedure **TransferFunds** that transfers a specified amount from one account to another, checking that the source account has sufficient balance before making the transfer.

Creating Table Accounts:

```
CREATE TABLE Accounts (  
  AccountID NUMBER PRIMARY KEY,  
  CustomerName VARCHAR2(50),  
  Balance NUMBER(10,2)  
);
```

Inserting Data:

```
INSERT INTO Accounts VALUES (101,'John',10000);  
INSERT INTO Accounts VALUES (102,'Alice',8000);  
  
COMMIT;
```

Creating Procedure:

```
CREATE OR REPLACE PROCEDURE TransferFunds(  
  p_FromAccount IN NUMBER,  
  p_ToAccount IN NUMBER,  
  p_Amount IN NUMBER  
)  
IS  
  v_Balance NUMBER;  
BEGIN  
  SELECT Balance  
  INTO v_Balance  
  FROM Accounts  
  WHERE AccountID = p_FromAccount;  
  
  IF v_Balance >= p_Amount THEN  
  
    UPDATE Accounts  
    SET Balance = Balance - p_Amount  
    WHERE AccountID = p_FromAccount;  
  
    UPDATE Accounts  
    SET Balance = Balance + p_Amount  
    WHERE AccountID = p_ToAccount;  
  
    COMMIT;  
  
  ELSE  
    DBMS_OUTPUT.PUT_LINE('Insufficient Balance');  
  END IF;  
END;  
/
```

Execute:

```
BEGIN
  TransferFunds(101,102,2000);
END;
/
```

Verify:

```
SELECT * FROM Accounts;
```

Output:

Query result					Script output	DBMS output	Explain Plan	SQL history
  Download 					Execution time: 0.016 seconds			
	ACCOUNTID	CUSTOMERNAME		BALANCE				
1	101	John		8000				
2	102	Alice		10000				